



TXW80X mass production and programming guide



Notice

Due to product version upgrades or other reasons, this document will be updated from time to time. Unless otherwise agreed, this document only

As a guide for use without any warranty.

Zhuhai Taixin
Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Confidentiality level	A	TXW80X mass production and programming guide	File No	TX-0000
Release date 2022-08-08			file version	V1.1

Revision history

date	Version	describe	Revised by
2022-08-08	V1.1	1. Add online burning description	TX
2022-07-22	V1.0	initial version	TX



Zhuhai Taixin Semiconductor Co., Ltd.
Zhuhai Taixin Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Copyright infringement will be investigated

Copyright © 2022 by Tai Xin All rights reserved

Confidentiality level	A	TXW80X mass production and programming guide	File No	TX-0000
Release date	2022-08-08		file version	V1.1
Table of Contents 1. Overview..... 1 2. Mass production programming interface 1 3. Mass production code generation 2 3.1. Code generation related configuration..... 2 3.1.1. Code space allocation file.....2 3.1.2. Code generation configuration file.....3 3.1.3. Factory parameter configuration file.....3 3.2. Code generation..... 5 3.3. Code Encryption 5 3.3.1. Solution without PSRAM6 3.3.2. Solution with PSRAM.....8 4. Chip FLASH startup and operation mechanism.....8 5. Code programming..... 9 5.1. MAC address rolling code burning..... 9 5.2. Code encryption and burning 10 5.3. Offline Programming (simple programmer: TXLINK-Lite).....10 5.4. Online programming.. 11 5.4.1. Simple programmer: TXLINK-Lite.....11 5.4.2. USB burning..... 13 5.4.3. CKLINK programming.....13				
		Zhuhai Taixin Semiconductor Co.,Ltd	3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City	

1 Overview

This article mainly describes the mass production and burning process of TXW80X, including the configuration, generation and encrypted code import of mass production bin files. Output, and mass production burning and other processes.

This document is mainly applicable to the following engineers:

- y Technical support engineer
- y Solution software development engineer

Product scope applicable to this document:

model	encapsulation	Package
TXW80X	QFN56/48/40/36/32	

2. Mass production programming interface

For mass production programming, a simple programmer: TXLink-Lite is used for programming. When programming the board, the board needs to be reserved offline. Programming interface: 3V3 (optional), CHIP_EN, PA8, PA9, GND (3V3 is fine if the programmer is not required to be powered) (not left), offline burning is possible under any circumstances.

The simple writer hardware is as follows:



The offline burning connection is as follows:

TXLink Lite	Target
VREF	3V3
nRST	CHIP_EN
GND	GND
HDA	PA8
HCK	PA9

3. Mass production code generation

3.1. Code generation related configurations

Currently, the SDK is built based on the integrated development environment CDK, and the code-related configuration mainly involves several files:

1. gcc_csky.ld: code space allocation file
2. makecode.ini: code generation configuration file
3. parameter_ui.setcfg: factory parameter configuration file

3.1.1. Code space allocation file

The space allocation file defaults to gcc_csky.ld. It mainly depends on the code space allocation. The code space needs to be

The starting position of the CodeExeAddr code in makecode.ini remains consistent.

```

16
17 /*****
18 * @file      gcc_csky.ld
19 * @brief     csky linker file
20 * @version   V1.0
21 * @date      02. June 2017
22 *****/
23 MEMORY
24 {
25     SRAM : ORIGIN = 0x20000000 , LENGTH = 0x48000 /* on-chip SRAM 288KB */
26     SRAM2 : ORIGIN = 0x20051000 , LENGTH = 0x1800
27     FLASH : ORIGIN = 0x18000000 , LENGTH = 0x100000
28     PSRAM : ORIGIN = 0x18800000 , LENGTH = 0x100000
29 }
30
31 __heap_size = 0x11000;
32
33 PROVIDE (__ram_end = 0x20048000);
34 PROVIDE (__heap_end = 0x20048000);
35
36 REGION_ALIAS("REGION_TEXT", FLASH);
37 REGION_ALIAS("REGION_TEXT2", SRAM2);
38 REGION_ALIAS("REGION_DATA2", SRAM2);
39 REGION_ALIAS("REGION_RODATA", FLASH);
40 REGION_ALIAS("REGION_DATA", SRAM);
41 REGION_ALIAS("REGION_BSS", SRAM);
42 REGION_ALIAS("REGION_INIT", SRAM);
43

```

Diagram illustrating memory regions and their aliases:

- SRAM : ORIGIN = 0x20000000 , LENGTH = 0x48000 /* on-chip SRAM 288KB */ → SRAM空间
- SRAM2 : ORIGIN = 0x20051000 , LENGTH = 0x1800 → SPI FLASH空间
- FLASH : ORIGIN = 0x18000000 , LENGTH = 0x100000 → PSRAM空间
- PSRAM : ORIGIN = 0x18800000 , LENGTH = 0x100000 → PSRAM空间
- REGION_ALIAS("REGION_TEXT", FLASH) → 代码空间
- REGION_ALIAS("REGION_TEXT2", SRAM2) → 数据空间
- REGION_ALIAS("REGION_DATA2", SRAM2) → 数据空间
- REGION_ALIAS("REGION_RODATA", FLASH) → 数据空间
- REGION_ALIAS("REGION_DATA", SRAM) → 数据空间
- REGION_ALIAS("REGION_BSS", SRAM) → 数据空间
- REGION_ALIAS("REGION_INIT", SRAM) → 数据空间

3.1.2. Code generation configuration file

The space allocation file is makecode.ini, which mainly configures the code running address and FLASH clock frequency offset. All configurations

All are hexadecimal numbers. Makecode.exe will parse the makecode.ini configuration, code bin and parameter configuration file generation

A code file with parameters and header information. The file name will automatically include the SDK version number, application type, date and customer.

Name the scheme, for example: txw80x_fpv_v2.0.0.7-17963_2022.5.24_TXW806_WAV_EVB.bin.

```

14 [SPI]
15 CodeCRC16=1 ; spi code crc enable
16 Flag=5A69 ; fixed
17 Version=0x0100 ; code version
18 CodeLoadToSramAddr=18000000 ; code load to sram start address
19 CodeExeAddr=18000000 ; code run start address
20 CodeAddrOffset=c00 ; code load from spi flash address Offset >= 1024 + param length
21 CodeLoadLen=2000 ; code load to sram length
22 SPI_SIZE=100000 ; spi flash size
23 SPI_CLK_MHZ=3c ; spi clk : 13Mhz
24 DriverStrength=1 ; DriverStrength : {0/1/2/3}, bigger sign stronger
25 PLL_SRC_MHZ=28 ;
26 PLL_EN=1 ;
27 DebugInfoEn=0 ;
28 AesEnable=0 ; aes encrypt enable
29 ; advance config
30 ReadCmd=6B ; spi read cmd index : normal(03)/fast(0B)/dual(3B)/quad(6B)/QPI(EB)
31 ReadCmdDummy=8 ; dummy clk
32 ClockMode=3 ; clock mode {0,3}
33 SampleDelay=5 ; 16bit sample delay cnt in spi clk divisor
34 WireModeWhenCmd=1 ; wire mode : {1/2/4}, 6 for 3wire mode
35 WireModeWhenAddr=1 ; wire mode : {1/2/4}, 6 for 3wire mode
36 WireModeWhenData=4 ; wire mode : {1/2/4}, 6 for 3wire mode
37 WireMode4Select=0 ; 4wire mode : D2/D3 io select{0,2}
38 WireMode4En=1 ; 4wire mode enable{0,1}
39 SpecSequenceEn=1 ; spec cmd squence before cmd read, eg.use for dual/quad cmd
40 SpecSequenceNumbers=3 ; spec cmd squence (format: cmd + dummy + data_lens + data) numbers
41 SpecSequence0=06000000 ; spec cmd "write enable "
42 SpecSequence1=31000102 ; spec cmd "qual mode enable for winbond "
43 SpecSequence2=05800101 ; spec cmd "wait busy "
44 SpecSequence3=38000000 ; spec cmd "QPI enable"
45 SpecSequence4=C0000133 ; spec cmd "Set Read Param"
46

```

代码运行地址, 和CDK工程Id文件保持一致

FLASH 时钟频率, 配置时同步配置SampleDelay = 480/clk/2+1

FLASH高级配置, 修改配置和原厂确认

[CodeLoadToSramAddr]: The SRAM address to which the code is loaded, generally consistent with CodeExeAddr

[CodeExeAddr]: The address where the code is to be run, generally consistent with CodeLoadToSramAddr

3.1.3. Factory parameter configuration file

TXW80X provides customers with highly personalized factory parameter configurations, and users can modify the parameters arbitrarily according to actual needs.

name and content.

The factory parameter configuration files are parameter_ui.setcfg and parameter.bincfg. When CDK is compiled, makecode will actively extract the parameter configuration and configure it into the code bin file.

This configuration file is configured through the TX_programmer.exe tool and can be imported and exported. The parameter format is as follows:

```

struct __param { uint16

    param_len; /* Number length */ uint16 param_flag; /* Magic Number:

    fix 0x2B1A */ struct __param_fn fn0;

    struct __param_fn fn1;

```

```

struct __param_fn fn_x; /* User can customize it*/

};

struct __param_fn { uint8 fn_num; /

    * Function code, it is recommended that users add and use it in descending order from 0xFE to avoid conflicts*/ /* This

size; uint8 buf[size]; /*      parameter configures size */ uint8

Parameter area*/

}

```

The parameter configuration interface is as follows:



Note: For solutions that require external PSRAM, parameter configuration area Fn1 must be configured.

Fn1 configuration description is as follows (hexadecimal):

flash_mode:

- 0: spi mode
- 1: Single line qspi mode
- 2: 4 line qspi mode
- 0xff: invalid error

psram_mode:

- 0: Abnormal (because psram does not exist in spi mode, it cannot be set to 0)
- 1: Single line qspi mode

2:4 line qspi mode

0xff: represents shared io with flash (flash and psram share io [except cs], so currently

flash must be set to 1 [single-wire mode] or 2 [4-wire mode])

[psram_io]: Configure psram io, including CS/CLK/D0/D1/D2/D3, in psram_mode=0xff

When , only cs is valid. When psram_mode=1 or 2, all pins are valid. 8bit represents a pin. The previous

3bit represents PA PB PC, etc., the last 5bit represents pin_num, for example: 0E represents PA14, 2E: represents: PB14, 3E: represents

PB30. For example: psram_io: 0EFFFFFFF, represents: CS:PA14, CLK, D0, D1, D2, D3 represents 0xff (invalid)

[psram_clk_mhz]: Set the clk frequency of psram, for example, set it to 3c, which represents (60*1M)Hz

[flash_clk_mhz]: Same calculation method as psram_clk_mhz

[psram_driver_strength]: Set the io driver capability of psram

[flash_driver_strength]: consistent with psram_driver_strength

[psram_sample_dly]: Set the dly value, it is recommended to use 0xff (automatic configuration)

[flash_sample_dly]: consistent with psram_sample_dly

[uart_tx]: Set the uart printing pin of the loader. The first 3 bits represent PA PB PC, etc., and the last 5 bits represent

pin_num

[uart_clk]: Set the baud rate of uart, the setting value *100 is the actual baud rate, for example, the setting value 2400: means yes

(9216*100) baud rate

3.2. Code generation

After the code generation related configuration is configured, directly CDK compile to generate the final bin file, makecode will

Two Bin files are generated at the same time, both in plain text. Among them, the bin file name with "_PSRAM" as the suffix is PSRAM.

The other Bin is used by the scheme without PSRAM, for example:

"txw80x_fpv_v2.0.0.7-17963_2022.5.24_TXW806_WAV_EVB": Use the solution without PSRAM

"txw80x_fpv_v2.0.0.7-17963_2022.5.24_TXW806_WAV_EVB_PSRAM": Solution with PSRAM

When using, **parameter configuration area Fn1 must be configured correctly.**

3.3. Code encryption

Code encryption needs to be completed on the chip that has already burned the encryption key (refer to Chapter 5.2).

3.3.1. Solution without PSRAM

The code is encrypted through the TX_programmer.exe tool. The code encryption steps are as follows:

1. Connect the board to the computer via USB (first erase the chip FLASH to ensure there is no code in the chip FLASH)

2. TX_programmer.exe communication protocol to "USB-ENCRYPT"

3. Select the compiled plain text firmware For example:

ÿtxw80x_fpv_v2.0.0.7-17963_2022.5.24_TXW806_WAV_EVBÿ

4. "Settings" -> "Chip Settings" -> "master boot" save

5. Click "Start" to start exporting the encryption code

6. In the Infor window, you can see the successful export of the encryption code. The encryption code will be added after the original file name.

ÿ“_gen”

The above operation diagram is as follows:





3.3.2. Solution with PSRAM

For the solution with PSRAM, you need to additionally execute "merge.bat" on the basis of the encryption code of the solution without PSRAM.

There are two main ways:

Option 1: Run the configuration tool command and select merge.bat ("Settings" -> "Chip Settings")

Option 2: Manually drag the encrypted Bin file to "merge.bat".

The encryption bin file name of the scheme with PSRAM has "_PSRAM" as the suffix.

4. Chip FLASH startup and operation mechanism

TXW80x supports dual code coexistence. When the chip is started, the code will be searched and loaded from FLASH by default. It will traverse the 2MB FLASH space to find the code in steps of 256KB. A maximum of two codes are allowed to exist at the same time. After the code search is completed, the latest code will be loaded, verified and executed. If there is a problem with the latest code (destroyed, dead, etc.), the chip will intelligently and automatically roll back to the old code execution to prevent code anomalies. Problems such as being unable to boot and upgrade normally occurred.

The dual code mechanism is usually reserved for scenarios where the firmware needs to be upgraded. However, if the FLASH size and burning time allow, the dual code can be built directly before leaving the factory. This method can improve product stability and service life (if FLASH abnormality causes A code exception). The dual code

space supports the following combinations:

1y256 KB + 256 KB

2512 KB + 512 KB

31024KB + 1024KB

If you need to build two codes during initial burning, you can use the BinScript tool that comes with the SDK or other tools.

With Merge Bin file, the empty space between the two codes is filled with 0xFF.

5. Code burning

5.1. MAC address rolling code burning

The programmer supports Mac address rolling code burning. Select "efuse" in "Settings"->"Chip Settings"->Mode

Then set the Mac address. If you need to save the Mac address to SPI Flash, check "Mac Save To Flash".

And configure the storage address (can be burned repeatedly); if not checked, the Mac address will be burned to efuse (only burning is allowed)

once). In addition, user-defined configurations are also burned into efuse (only allowed to be burned once). The configuration interface is as follows:



5.2. Code encryption and burning

For program codes that require encryption protection, burning requires configuring a key file. Only the key file matches the encryption code.

The configuration code can run normally. The key file is: efuse_key_***.dll, the configuration is as follows:



5.3. Offline programming (simple programmer: TXLINK-Lite)

Programmer PC software

1. TX_programmer.exe communication protocol to "USB-PROGLITE"
2. Connect the simple writer to the computer via USB
3. Select the file to be burned (plain text or cipher text)
4. Click "Start" to start updating the programmer
5. After the update is successful, connect the chip Socket to the programmer for manual programming or machine programming.

The schematic diagram of the programmer update operation is as follows:



5.4. Online programming

5.4.1. Simple programmer: TXLINK-Lite

Online programming with a simple programmer (TXLINK-Lite) supports up to 1-to-18 programming. Burn various detailed parameter configurations

The settings are consistent with offline burning.

Please pay attention to channel binding when burning online and dragging and burning. The channel will be bound to the device inserted first, and the newly inserted channel will be bound first.

The old programming status information of the channel will be cleared;

When the online programming programmer is removed, the channel will flash to prompt the device to be removed, and the programming status information of the channel will be retained.

Display to confirm the programming failure and success status.

The configuration for online burning and one-to-many burning is as follows:





5.4.2. USB burning

For solutions that support USB or for chips that are burned for the first time, you can use PC tools to directly burn them online. The steps are as follows:

as follows:

1. TX_programmer.exe communication protocol to "USB-BOOT"
2. Connect the target board to the computer via USB
3. Select the file to be burned (plain text or cipher text)
4. Click "Start" to start programming the target chip
5. After successful burning, power off and restart the code.

5.4.3. CKLINK burning

For details, see Chapter 2.4 of "SDK Quick Start Manual.pdf".